

Smart MCQ Solver

Deep Learning & Generative AI — Project Report

-Shreyash Srivastava
22f3001070

1. Abstract / Executive Summary

This report documents the design, implementation, and evaluation of three modelling approaches built for the Kaggle “smart-mcq-solver-challenge”, a multiple-choice question answering task in which each question provides a prompt and five candidate options (A–E), exactly ones of which is correct. Performance is measured with Mean Average Precision at 3 (MAP@3), which rewards ranking the correct option within the top three predictions.

Three progressively more sophisticated systems were built and tracked end-to-end in Weights & Biases (W&B): (1) a classical TF-IDF + Logistic Regression baseline, (2) a custom feed-forward neural network (MLP) built from scratch in PyTorch, and (3) a fine-tuned RoBERTa-base transformer trained as a 5-way sequence classifier. The fine-tuned RoBERTa-base model achieved the best validation performance (MAP@3 = 0.9946), followed by the TF-IDF + Logistic Regression baseline (MAP@3 = 0.9692); the from-scratch model underperformed both (MAP@3 = 0.9325), showing that architectural depth alone does not guarantee better results without adequate training regime (mini-batching, epochs, regularisation).

2. Introduction

2.1 Problem Statement

The competition poses a multiple-choice QA problem: each record has a natural-language prompt and five candidate answers (A–E) with one ground-truth label. The task is to output, per question, a ranked list of up to three options such that the correct answer appears as early as possible.

2.2 Project Objective

The objective was to build and compare solutions of increasing modelling complexity — a classical bag-of-words baseline, a custom neural network implemented from first principles, and a fine-tuned pre-trained transformer — to understand accuracy, efficiency, and implementation trade-offs, while gaining hands-on experience with scikit-learn, PyTorch, and the Hugging Face ecosystem, with full experiment tracking in W&B.

2.3 Report Structure

Section 3 covers the dataset and preprocessing. Section 4 covers tokenization strategy. Section 5 details each model's architecture and training configuration. Section 6 presents comparative results with W&B metrics. Section 7 concludes with learnings, challenges, and future work; Section 8 lists references.

3. Dataset & Preprocessing

3.1 Dataset Description

The dataset is supplied by the Kaggle competition as a labelled training set (loaded via `load_data()`), with each row containing a question prompt, five option columns (A–D... E), and an answer column giving the correct letter. A held-out, unlabelled test set is provided for leaderboard submission.

DATASET OVERVIEW

=====

Train shape : (2000, 10)

Test shape : (500, 8)

Columns : ['id', 'prompt', 'A', 'B', 'C', 'D', 'E', 'answer', 'text', 'label']

First 3 rows:

	id	prompt	A	B	C	D	E	answer	text	label
0	1	pick the best possible answer: what is martin ...	martin heidegger believes that humans exist wi...	martin heidegger believes that humans do not e...	martin heidegger does not believe in the exist...	martin heidegger believes that the relationshi...	martin heidegger believes that time is an illu...	B	pick the best possible answer: what is martin ...	1
1	2	what is accelerator-based light-ion fusion?	accelerator-based light-ion fusion is a techni...	accelerator-based light-ion fusion is a techni...	accelerator-based light-ion fusion is a techni...	accelerator-based light-ion fusion is a techni...	accelerator-based light-ion fusion is a techni...	A	what is accelerator-based light-ion fusion? \nA...	0
2	3	determine the correct option: what is the term...	blueshifting	redshifting	reddening	whitening	yellowing	C	determine the correct option: what is the term...	2

3.2 Exploratory Data Analysis

Standard EDA for this task type includes distribution of prompt/option lengths, class balance across A–E, and duplicate-question checks; histogram and bar-chart visualisations of these statistics should be inserted here from the notebook, as they are not part of the raw code/log output captured in this report.

3.3 Data Preprocessing

A uniform text-cleaning function was applied to the prompt and all five option columns prior to feature extraction and tokenization, and missing values were filled with empty strings to avoid errors during vectorisation:

```
def clean(text):
    text = str(text).lower()      # Uniforming case
    return ' '.join(text.split()) # Removing extra whitespace
```

For the classical and custom models, each question was expanded into five (prompt, option) pairs labelled 1 (correct) or 0 (incorrect) — turning 5-way ranking into pointwise binary scoring, re-aggregated into a ranking at inference time. For the transformer, prompt and all five options were concatenated into one text block and modelled directly as 5-way classification.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection & Justification

Two representations were used: (a) for the classical and custom models, a scikit-learn TfidfVectorizer with unigrams + bigrams (ngram_range=(1,2)), a 50,000-feature vocabulary cap, and English stop-word removal; (b) for the transformer, the RoBERTa byte-level tokenizer (AutoTokenizer.from_pretrained("roberta-base")) associated with the pre-trained roberta-base checkpoint (Liu et al., 2019). TF-IDF is fast, interpretable, and needs no GPU, capturing locally discriminative n-grams; the RoBERTa tokenizer instead captures sub-word structure, word order, and contextual semantics that a bag-of-words model cannot, which matters when options are semantically close.

4.2 Implementation Details

Transformer tokenization was implemented via the Hugging Face datasets library with `batched=True`, `truncation=True`, and `padding="max_length"` (`max_length=512`), and the tokenized columns were formatted directly as PyTorch tensors (`input_ids`, `attention_mask`, `labels`).

4.3 Experimental Comparison

The same fitted TF-IDF vectoriser was reused unchanged for both the Logistic Regression baseline and the custom neural network, isolating the effect of model architecture from feature representation; the transformer's tokenizer pipeline was kept independent since it needed order-preserving, fixed-length tensors compatible with the pre-trained checkpoint.

5. Modelling & Experimentation

All models were evaluated on an identical, leakage-safe split: question IDs (not individual rows) were split 80/20 (`test_size=0.2`, `random_state=42`), keeping all five options of a question on the same side of the split.

5.1 Model 1: Fine-Tuned Transformer (RoBERTa-base)

Architecture: A pre-trained roberta-base checkpoint (Liu et al., 2019) was fine-tuned end-to-end as a 5-way sequence classifier via `AutoModelForSequenceClassification(num_labels=5)`. RoBERTa is a BERT-derived, encoder-only transformer pre-trained with dynamic masked-language-modelling and byte-level BPE tokenization, without next-sentence prediction.

Salient points: Self-attention across 12 encoder layers builds a joint contextual representation of the prompt and all five options, capturing prompt–option relationships a bag-of-words model cannot. A new classification head was randomly initialised on top of the pre-trained encoder (confirmed by “MISSING” keys at load time).

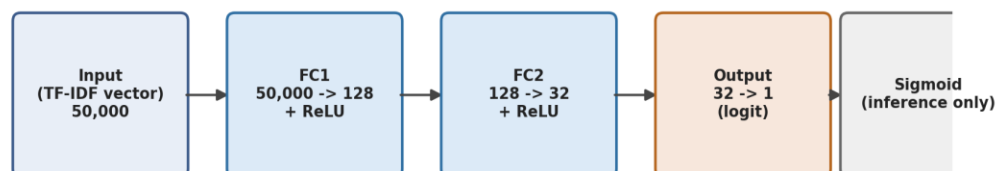
Fine-tuning strategy: The full model was fine-tuned with no frozen layers, using the Hugging Face Trainer, learning rate $2e-5$, weight decay 0.01, batch sizes 8 (train) / 16 (eval), 3 epochs, evaluation/checkpointing per epoch, and the best checkpoint reloaded at the end. No quantization was applied (`fp16=False`).

Epoch	Training Loss	Validation Loss	Val. Accuracy
1	2.8809	2.3349	0.565
2	0.7351	0.4137	0.960
3	0.1157	0.0977	0.990

5.2 Model 2: Custom Deep Learning Model (Feed-Forward MLP)

Architecture: A 3-layer multilayer perceptron built from scratch in PyTorch on the same 50,000-dimensional TF-IDF features ($50,000 \rightarrow 128 \rightarrow 32 \rightarrow 1$), with ReLU activations and a single output logit scored via `BCEWithLogitsLoss`:

Custom Feed-Forward Neural Network (MLP) Architecture



Salient points: The network progressively compresses the sparse, high-dimensional TF-IDF input through two ReLU-activated dense layers, in principle allowing it to learn non-linear feature interactions the linear baseline cannot. It was trained with the Adam optimiser (`lr=0.001`) for 10 full-batch epochs (a `DataLoader/TensorDataset` was imported

but mini-batching was not actually used); training loss decreased slowly from 0.6562 (epoch 1) to 0.6169 (epoch 10), indicating the model was still early in convergence when training stopped — the most likely cause of its comparatively weaker MAP@3 versus the linear baseline (Section 6).

5.3 Model 3: TF-IDF + Logistic Regression (Classical Baseline)

As explicitly requested, the third model uses our own chosen method rather than an RNN: a classical TF-IDF + Logistic Regression baseline ($C=1.0$, $\text{max_iter}=1000$), scoring each (prompt, option) pair's 50,000-dim TF-IDF vector for probability of correctness. This linear, convex model trains in seconds on CPU and provides a fast, robust reference point against which the two deep-learning approaches above are measured. For each question the five option-probabilities were reshaped into an $(n_questions \times 5)$ score matrix and the top-3 options taken as the MAP@3 prediction.

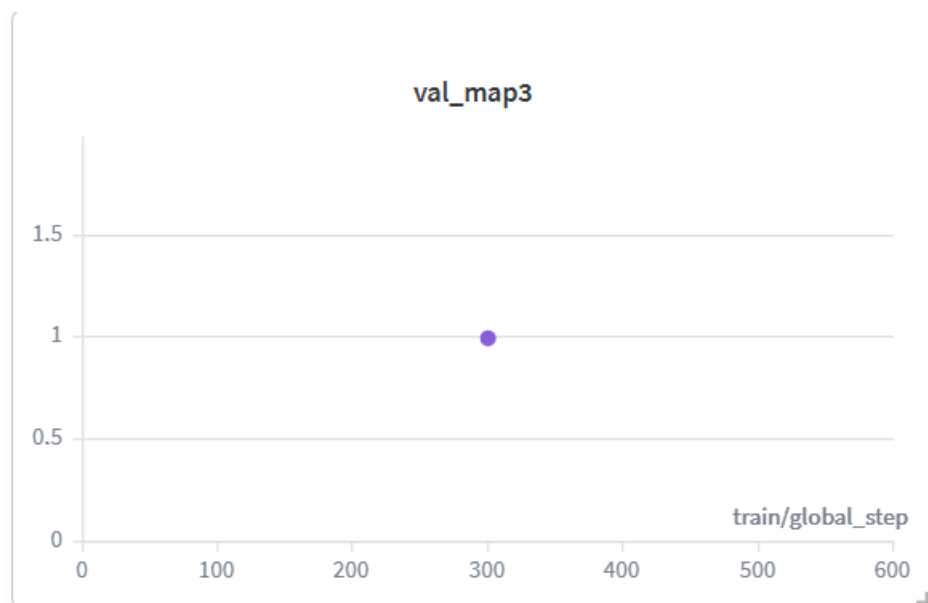
6. Performance & Comparative Analysis

6.1 Evaluation Metrics

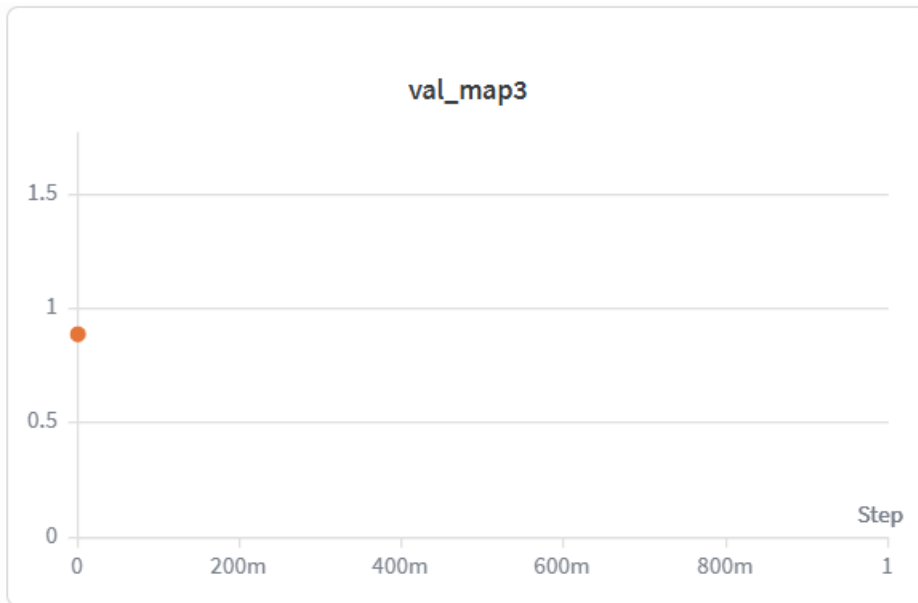
The competition's primary metric is Mean Average Precision at 3 (MAP@3). For the transformer, classification accuracy (argmax) was additionally tracked per epoch as an easier-to-interpret secondary signal.

6.2 Training Performance (W&B)

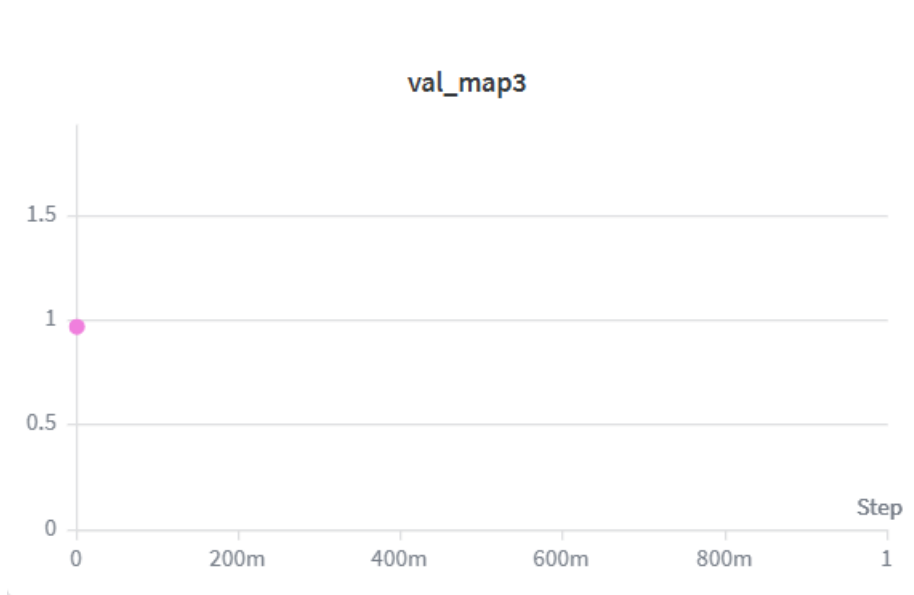
All three runs (tfidf-lr, nn-scratch, roberta-base) were logged to the same W&B project (smart-mcq-solver, entity 22f3001070-dlgenai-project-T2). The RoBERTa run logged per-epoch loss/accuracy curves via `report_to="wandb"`; the nn-scratch logged a smoothly decreasing training loss over 10 epochs; the TF-IDF baseline logged a single final `val_map3` value since it is not trained iteratively.



RoBERTa



nn-scratch



Tf-idf with logistic regression

6.3 Comparative Report

Model	Approach	Val. MAP@3	Val. Accuracy	Approx. Training Time
TF-IDF + Logistic Regression	Classical ML baseline	0.9692	—	< 1 min
Custom NN (from scratch)	3-layer feed-forward NN	0.9325	—	~1–2 min (10 epochs)
RoBERTa-base (fine-tuned)	5-way transformer classifier	0.9946	0.990	~5 min (3 epochs, GPU)

Discussion: RoBERTa-base achieves the best performance (MAP@3 = 0.9946) via contextual, pre-trained representations and full fine-tuning, at the cost of higher compute. The TF-IDF + Logistic Regression baseline is a strong, very cheap alternative (0.9692), attractive under tight compute/latency budgets. The custom MLP (0.9325)

underperformed the simpler linear baseline despite greater depth, most likely due to the limited epoch budget, full-batch (rather than mini-batch) updates, and absence of regularisation — all addressable in future work.

6.4 Kaggle Performance

364	SHREYASH_Kaggle_IND		0.75394	18
-----	---------------------	---	---------	----

7. Conclusion & Future Work

7.1 Key Learnings

- A well-tuned classical baseline (TF-IDF + Logistic Regression) remains highly competitive and extremely cheap for short-text classification tasks such as MCQ answer scoring.
- A custom neural network is not guaranteed to outperform a simpler model — training regime (epochs, batching, regularisation) matters as much as architectural depth.
- Fine-tuning a pre-trained transformer (RoBERTa-base) with few epochs and a modest learning rate ($2e-5$) yielded the largest gain, confirming the value of transfer learning even with limited labelled data.
- Leakage-free splitting by question ID (not row) was essential given the pointwise formulation used for the classical/custom models.

7.2 Areas for Improvement

- Introduce proper mini-batching, more epochs, and dropout/weight decay for the custom MLP to give it a fairer comparison.
- Add a dedicated RNN-based model (e.g., bi-directional LSTM/GRU) as a further architectural comparison point.
- Explore ensembling (averaging/stacking) TF-IDF and RoBERTa option scores to combine lexical and contextual strengths.
- Apply data augmentation and a systematic hyperparameter search (learning rate, epochs, weight decay) for the transformer.

8. References

1. Liu, Y., Ott, M., Goyal, N., et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.
2. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT.
3. Hugging Face. transformers and datasets library documentation. <https://huggingface.co/docs>
4. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825–2830.
5. Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS.
6. Biewald, L. (2020). Experiment Tracking with Weights and Biases. <https://www.wandb.com/>